VITYARTHI – PROJECT

Project title : Multi-Unit Command-Line Converter

Course title : Programming in Java

Course code : CSE 2006

Course type : Flipped course

Course credits : 3

Professor : Dr. Rizwan Ur Rahman

Slot : F11 + F12

Submitted by : Raghul.S.K

Registration number : 25BAI10288

Submission date : 08/09/2026

VITYARTHI – PROJECT

INTRODUCTION :

The Multi-Unit Command-Line Converter is an efficient and lightweight software utility designed to solve common measurement conversion challenges across diverse physical, digital, and financial domains. In technical education, software engineering, and scientific computation, values are frequently presented in disparate unit standards—such as metric versus imperial scales, legacy data storage formats, or international currencies. Converting these values manually or relying on resource-heavy web converters introduces friction and human error.

The Multi-Unit Command-Line Converter provides a unified, terminal-based platform supporting dual interaction paradigms: direct command-line arguments (CLI) for single-step conversions and an interactive Read-Eval-Print Loop (REPL) shell for multi-query execution. Developed in Java, this project showcases core object-oriented principles, stream-based string parsing, modern switch expressions, and regular expressions to create an offline, dependable, and high-performance utility.

PROBLEM STATEMENT :

Engineering students, software developers, and systems analysts often encounter measurement values across incompatible systems—spanning metric and imperial engineering scales, time resolutions, storage capacities, and baseline financial conversions. Without a centralized, fast, and offline utility, navigating across fragmented conversion tables or opening bulky web interfaces is inefficient. The Multi-Unit

Command-Line Converter provides a robust, terminal-based tool that eliminates cross-domain conversion errors, standardizes natural language inputs, and yields instant, verified outputs.

VITYARTHI – PROJECT

PROJECT OBJECTIVES :

- Unified Multi-Domain Engine: To implement a single Java program supporting 7 distinct measurement categories through a centralized scaling factor architecture and dedicated affine transformation formulas.
- Natural Language Input Normalization: To build an input preprocessor (normalize) that standardizes plural forms, informal abbreviations, and colloquial aliases into canonical tokens.
- Category Safety Enforcement: To prevent mathematically invalid conversions across incompatible physical dimensions (such as length to mass) using category validation logic (getGroup).
- Dual Execution Workflows: To implement both non-interactive CLI argument processing for automation scripts and a continuous interactive shell (interactiveShell) for console users.
- Defensive Error Handling: To prevent application crashes through structured exception handling (try-catch) and clear feedback for non-numeric or missing inputs.
- Empirical Testing & Verification: To test conversion factors against international standard metrics and verify numerical precision across boundary conditions.

FUNCTIONAL REQUIREMENTS :

1. User-System Interaction

- The system executes conversions directly when supplied with terminal arguments (<value> <from> <to>).
- The system automatically enters an interactive REPL shell when launched without arguments.

VITYARTHI – PROJECT

- An integrated help command (help or -l) outputs all supported units and syntax structures.
- The shell terminates gracefully upon receiving quit or exit.

2. Data Input & Tokenization

- Accepts input arguments in two distinct conversational formats:
  - 3-token format: <value> <from> <to> (e.g., 50 km m)
  - 4-token format: <value> <from> to <to> (e.g., 10 miles to km)
- Isolates input components through whitespace regex tokenization (\\s+).

3. Input Normalization

- Strips leading and trailing whitespace and converts string tokens to lowercase.
- Maps synonyms, plural variations, and colloquial designations to standardized unit identifiers (e.g., dollars, dollar, bucks → usd; kilometers, kilometer → km; celsius → c).

4. Data Processing & Conversion

- Affine Temperature Conversions: Uses dedicated transformation logic (runTemp) for non-linear scales (C, F, K), routing through Celsius as an intermediate state.
- Linear Scalar Conversions: Uses scaling ratios anchored to base units via the formula:

$$\textbf{Result} = \frac{\textbf{Value} \times \textbf{Source Factor}}{\textbf{Target Factor}}$$

- Group Consistency Check: Evaluates whether source and target units belong to matching categories (len, mass, time, speed, data, curr) before running calculations.

5. Output Formatting

- Outputs linear conversions formatted to 4 decimal places with capitalized unit tags.
- Outputs temperature conversions formatted to 2 decimal places with capitalized unit tags.
- Displays explicit diagnostic messages when incompatible categories or malformed inputs are detected.

NON-FUNCTIONAL REQUIREMENTS :

1. Performance

- Executes computations and renders output in under 50 milliseconds per transaction.
- Minimal JVM startup overhead and near-zero memory footprint during interactive sessions.

2. Security

- Completely local execution without external network telemetry, outbound requests, or tracking.
- Safe parsing of string inputs to prevent buffer overruns or unhandled exception termination.

3. Usability

- Minimalist syntax footprint with clear prompt guides (>) and contextual error notifications.
- Support for natural input syntax reduces rigid command-line memorization.

4. Reliability & Availability

- Runs entirely offline without relying on network bandwidth or remote web services.

- Defensive parsing ensures invalid commands do not terminate active REPL sessions.

5. Maintainability

- Modular static method architecture (Process, normalize, getFactor, getGroup, runTemp).
- Implementation of modern Java syntax features (arrow switch expressions) for cleaner code maintenance.

6. Resource Efficiency

- Requires standard JVM runtime environments with minimal memory allocation (< 15 MB heap).
- No heavy third-party dependencies or external JAR files.

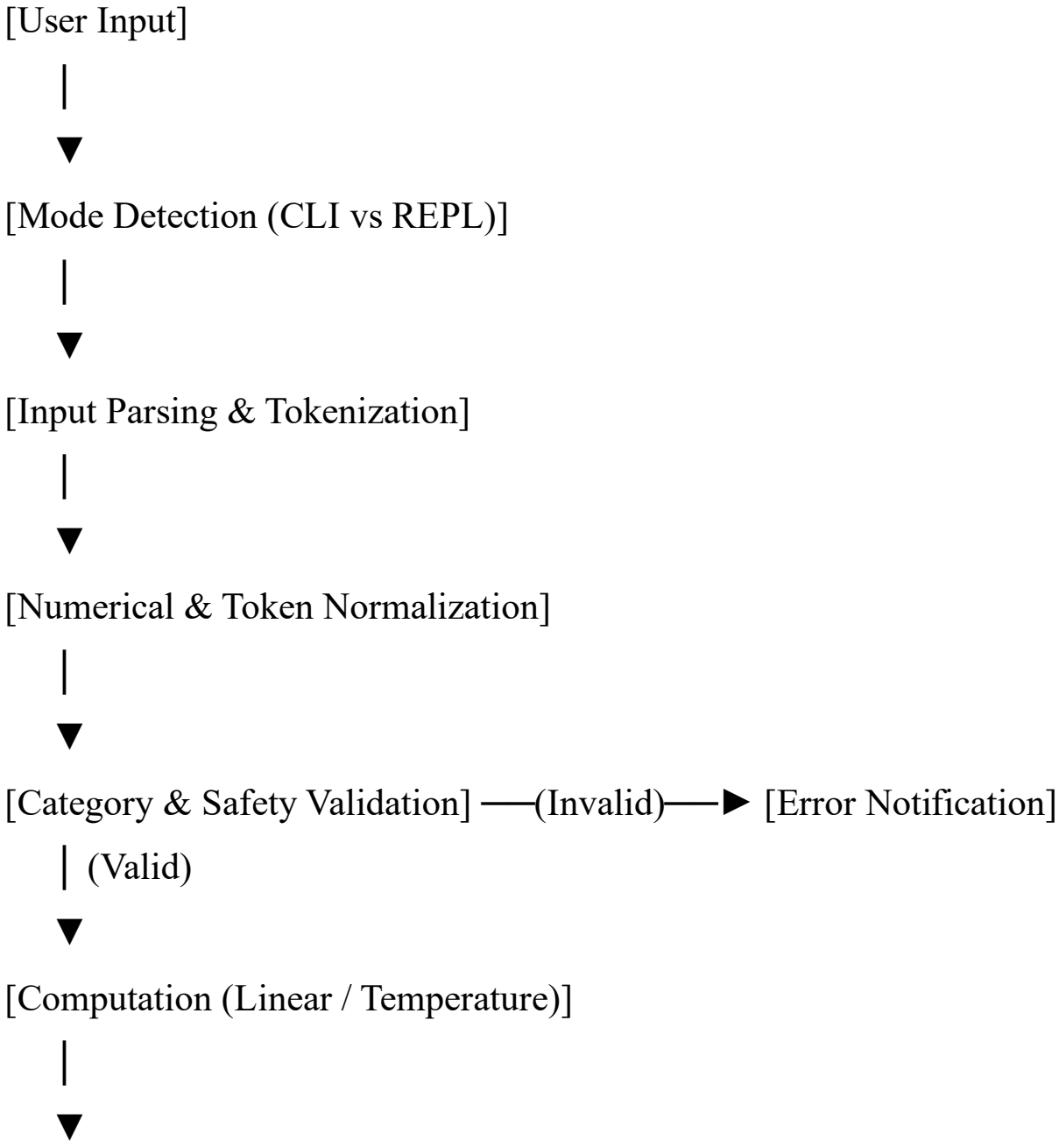
SYSTEM ARCHITECTURE :

- Presentation Layer (CLI / REPL): Handles command-line arguments through args[] or interactive keyboard stream capture via Scanner(System.in).
- Preprocessing & Normalization Layer: Sanitizes raw string tokens through lowercase mapping and alias resolution (normalize).
- Validation & Routing Layer: Verifies numerical validity via Double.parseDouble, checks unit compatibility via getGroup, and routes inputs between linear and temperature pathways.
- Computation Engine: Evaluates unit scaling factors (getFactor) or executes temperature conversions (runTemp).
- Output Renderer: Formats and prints calculated results or error notices directly to System.out.

DESIGN DIAGRAMS :

VITYARTHI – PROJECT

1. Workflow



2. Data Flow Architecture

- Input Stage: Captures raw tokens representing magnitude, source unit, and destination unit.

- Validation Stage: Parses string values into floating-point numbers (double); non-numeric inputs trigger graceful error catch blocks.
- Classification Stage: Identifies unit domain via regex (len, mass, time, speed, data, curr, or temperature flags).
- Execution Stage: Computes derived values via scaling factors relative to base standard dimensions ($m, kg, s, m/s, B, USD$).
- Presentation Stage: Formats numerical precision and prints uppercase unit tags to standard output.

3. Component Breakdown

- Java (Main Driver): Handles program entry, mode delegation, and command-line parsing.
- interactiveShell: Manages the REPL loop, prompt interface, and session lifecycle.
- Process: Coordinates validation, routing, calculation, and output display.
- normalize: Normalizes string aliases, abbreviations, and plurals into unified tokens.
- getGroup & getFactor: Group identification engine and conversion lookup table.
- runTemp: Dedicated transformation engine for thermodynamic scales.

DESIGN DECISIONS & RATIONALE :

1. Programming Language: Java

- *Rationale:* Java provides strong type safety, cross-platform portability across JVM runtimes, and modern structural syntax (like switch expressions) suited for clean lookup implementations.

2. Interface Paradigm: Command-Line Interface & REPL

- *Rationale:* Developers and technical users benefit more from keyboard-driven, scriptable terminal execution than from graphical interfaces that consume unnecessary system resources.

3. Base-Unit Scaling Factor Pattern

- *Rationale:* Rather than creating $N \times (N - 1)$ individual conversion formulas, mapping each unit to a single universal category base unit (e.g., all lengths to meters) reduces algorithmic complexity from $O(N^2)$ to $O(N)$.

4. Dual-Route Temperature Computation

- *Rationale:* Thermodynamic temperature systems involve offset baselines (affine transformations) rather than simple scalar multiplication, requiring separate algorithmic handling.

5. In-Memory Execution Architecture

- *Rationale:* Unit conversion calculations are transient; in-memory processing guarantees low latency without introducing disk I/O overhead.

IMPLEMENTATION DETAILS :

- Language: Java (JDK 17+)
- Development Environment: Visual Studio Code / Eclipse / IntelliJ IDEA / Terminal CLI
- Source File: Java.java
- Primary Class: Java

VITYARTHI – PROJECT

1. main(String[] args): Routes execution between direct one-line CLI conversion and interactiveShell().
    - interactiveShell(): Manages user session loops and natural conversational command inputs.
    - Process(String number_, String from_, String target_): Performs parsing, validation, computation, and output display.
    - normalize(String txt): Uses pattern switches to map multi-word/plural inputs into base keys.
    - getFactor(String u): Returns scalar conversion constants relative to standard base units.
    - runTemp(double v, String f, String t): Applies conversion formulas between Celsius, Fahrenheit, and Kelvin.

TESTING APPROACH :

1. Unit Testing: Validated individual methods (getFactor, normalize, runTemp) against standard mathematical references.

2. Domain Boundary Testing: Verified edge-case conversions across extreme units (e.g., millimeters to kilometers, bytes to terabytes).

3. Affine Temperature Verification: Tested freezing points, boiling points, and absolute zero ($0\ \text{K} = -273.15°\text{C} = -459.67°\text{F}$) to confirm formula precision.

4. Negative & Error Path Testing: Evaluated invalid inputs, such as non-numeric strings (abc km m), unmapped unit tokens, and incompatible cross-category requests (10 m to kg).

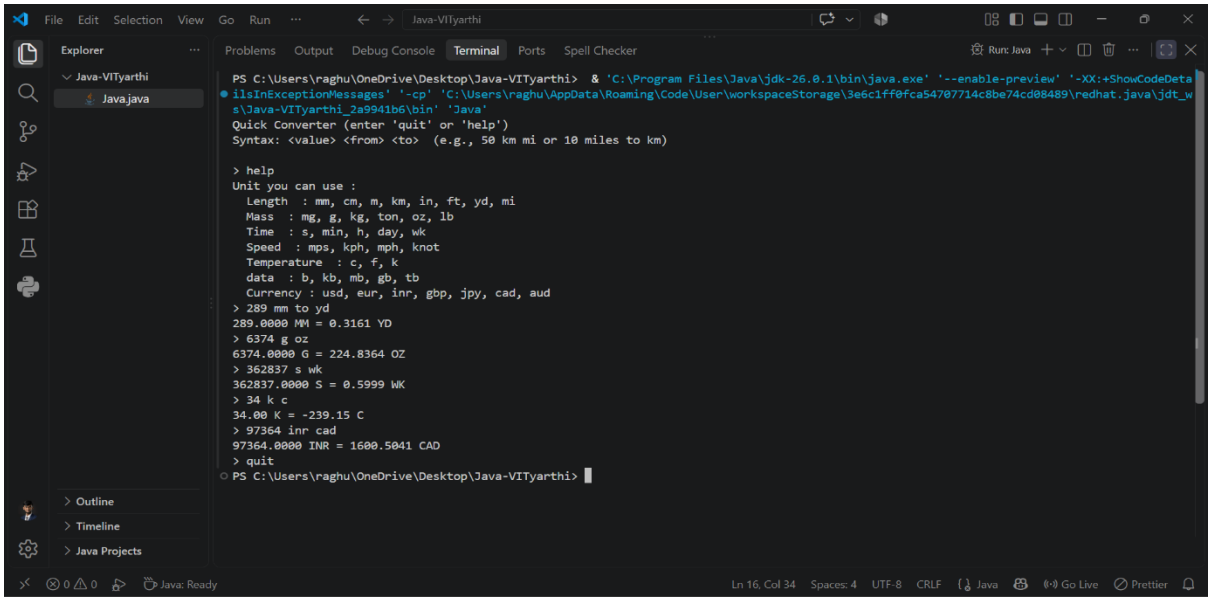
5. Shell Usability & Flow Testing: Verified that REPL edge cases, including whitespace variances, empty lines, uppercase variations, and exit commands, behave predictably.

VITYARTHI – PROJECT

CHALLENGES FACED :

- Handling Non-Linear Transformations: Unlike other measurement units that scale through simple scalar ratios, temperature requires offset baselines. This was resolved by designing an independent intermediate routing mechanism (runTemp).
- Input Variability & Natural Language: Users supply variations such as "miles", "mile", "mi", or colloquialisms like "bucks". An extensive normalization table was developed using pattern-matching switch logic.
- Preventing Invalid Cross-Category Conversions: Guarding against dimensionally invalid conversions (e.g., converting distance to speed) required regex-based category validation via getGroup().
- Regex Domain Boundary Omission: Testing revealed that the mi (mile) unit token was defined in getFactor() and normalize(), but omitted in the getGroup() regular expression pattern. This was identified and resolved by updating the regular expression matcher.

OUTPUT :



VITYARTHI – PROJECT

LEARNINGS AND KEY TAKEAWAYS :

- Scalar Dimensional Modeling: Gained hands-on experience reducing system complexity by anchoring unit calculations to base standards rather than hardcoding point-to-point mappings.
- Modern Java Paradigms: Applied newer Java language features, including modern switch expressions and lambda-style arrows, to write concise and readable conversion tables.
- Resilient Parsing: Learned the importance of defensive programming—specifically, how string sanitization, tokenization checks, and exception management preserve console stability.
- CLI UX Optimization: Realized that small design adjustments, such as supporting natural phrasing (10 miles to km) alongside brief abbreviations (10 mi km), significantly improve command-line usability.

FUTURE ENHANCEMENTS :

- Dynamic REST API Currency Integration: Replace static baseline currency multipliers with real-time exchange rates fetched via an external financial API.
- Additional Engineering Dimensions: Add scientific domains such as Pressure (Pascal, bar, psi), Energy/Work (Joule, calorie, kWh), and Power (Watt, horsepower).
- Arbitrary Precision Support: Integrate BigDecimal for high-precision scientific or financial environments where IEEE 754 floating-point inaccuracies are unacceptable.
- Terminal Autocomplete & Rich TUI: Introduce tab-completion and terminal formatting using frameworks like Picocli or Lanterna for a richer command-line experience.

VITYARTHI – PROJECT

REFERENCES :

- National Institute of Standards and Technology (NIST) – *Guide for the Use of the International System of Units (SI)*.
- Oracle Java Documentation – *Switch Expressions, Regular Expressions, and Scanner API*.
- IEEE 754-2019 – *Standard for Floating-Point Arithmetic*.